L06: Algorithm 4.1 — The Event Loop in Detail

Simulation Without a Black Box

Outline

Learning Objectives

Pseudocode for the Event Loop

Event Types and State Transitions

Scheduling Events from Within Routines

Stopping Conditions

Step-by-Step Trace: First Three Events

Simultaneous-Event Convention

Key Takeaways

Learning Objectives

By the end of this lecture you will be able to:

- Read and explain the pseudocode for Algorithm 4.1 (the event loop).
- Describe what each of the four event types (arrival, service-begin, service-end, departure) does to system state.
- Explain how event routines schedule new events.
- Choose and justify a stopping condition for a given study.
- Trace the first three events of the M/M/1 example step-by-step.
- Apply the departure-before-arrival tie-breaking convention.

Algorithm 4.1: The Event Loop

```
Initialize:
  t <- 0
  State <- initial_state()
  Calendar.insert(Arrival, t=first_arrival)

WHILE stopping_condition is False:
  (t_e, event_type, data) <- Calendar.remove_min()
  t <- t_e                    # advance clock
  update_statistics(t, State)

  IF   event_type == ARRIVAL:
  ↪  handle_arrival(data)
  ELIF event_type == SVC_END:
  ↪  handle_svc_end(data)
  ELIF event_type == DEPARTURE:
  ↪  handle_departure(data)
  ELIF event_type == SIM_END:      break

Report output statistics
```

Key design points

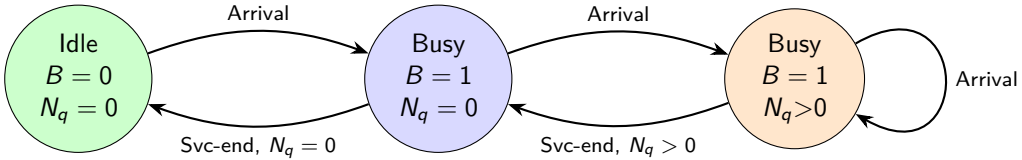
- Statistics are updated *before* executing the event (time-average area).
- Every handler may insert zero or more new events into the calendar.
- The loop is data-driven: no explicit branching on time.
- SIM_END is itself an event — insert at initialization.

Four Event Types: What Each Does to State

Event	N_q	B	**Schedules next**	**Side effect**
Arrival	$+1$ if $B{=}1$; else 0	1 if idle	Next arrival; svc-end if idle	Entity joins queue or server
Service-begin	-1	1	Service-end at $t + S_i$	Entity moves from queue to server
Service-end	—	0 if $N_q{=}0$; else 1	Departure; svc-begin if $N_q > 0$	Entity exits server
Departure	—	—	(none)	Entity leaves system; record W_i

Note: in many formulations service-begin and departure are merged into service-end. Keeping them separate makes the logic clearer during tracing.

State Transition Diagram



Scheduling New Events from Within Event Routines

Arrival handler:

```
def handle_arrival(entity):
  N_q += 1
  if not B:            # server idle
    B = True
    N_q -= 1
    S = sample_service()
    Calendar.insert(
      SVC_END, t + S, entity)
  # schedule next arrival
  A = sample_interarrival()
  Calendar.insert(ARRIVAL, t + A)
```

Service-end handler:

```
def handle_svc_end(entity):
  entity.depart_time = t
  record_sojourn(entity)
  if N_q > 0:
    next_entity = queue.pop()
    N_q -= 1
    S = sample_service()
    Calendar.insert(
      SVC_END, t + S, next_entity)
  else:
    B = False    # server goes idle
```

Every handler leaves the calendar consistent: no dangling references.

Stopping Conditions

Condition	**Use case**	**Implementation**
Time limit T	Steady-state study; fixed shift	Insert SIM_END at $t = T$
Customer count N	Terminating study; fixed demand	Counter in departure handler
Empty calendar	Terminating study; all served	`if Calendar.empty(): break`
Convergence criterion	Rare; pilot runs only	Check output variance each step

Common mistake

Using an empty calendar as the stopping condition for a steady-state study. If arrivals are generated only when a previous arrival fires, the calendar empties when the last arrival's chain ends — the simulation terminates too early.

Step-by-Step Trace: M/M/1 Example

Parameters: $\lambda = 1/\text{min}$, $\mu = 1.25/\text{min}$. Given: arrival times $A_1 = 1.2$, $A_2 = 3.7$; service times $S_1 = 0.6$, $S_2 = 1.1$.

Step	Event	t	N_q	B	Action	Calendar
Init	—	0.0	0	0	Insert ARRIVAL@1.2, SIM_END@60	{1.2-A, 60-END}
1	ARRIVAL	1.2	0	1	C1 seizes server; insert SVC_END@1.8; insert ARRIVAL@3.7	{1.8-SE, 3.7-A, 60}
2	SVC_END	1.8	0	0	C1 departs; $W_1 = 0.6$; $N_q = 0$, server idle	{3.7-A, 60}
3	ARRIVAL	3.7	0	1	C2 seizes server; insert SVC_END@4.8; insert ARRIVAL@next	{4.8-SE, ..., 60}

No waiting so far: both customers found the server idle. Congestion appears when $N_q > 0$.

Simultaneous Events: Departure Before Arrival

Convention (Algorithm 4.1, §4.3)

When a departure and an arrival are scheduled at the same simulated time t: process the **departure first**, then the arrival.

Why this matters:

- If arrival fires first: arriving customer sees a busy server, joins queue, then the departing customer frees the server — unnecessarily routes through queue.
- If departure fires first: server becomes idle before arrival fires — arriving customer seizes server immediately.

Effect on statistics: W_q differs between conventions. Pick one and apply it *consistently* throughout the study. **In Python heapq:** ties in event time are broken by the second tuple element. Assign numeric priorities: Departure $= 0$, Arrival $= 1$.

Key Takeaways

- Algorithm 4.1 is simple: loop over (remove, advance, execute, schedule).
- Each event type has a precise, deterministic effect on the state variables.
- Event handlers schedule new events — the calendar grows and shrinks dynamically.
- Stopping conditions must match the study type (terminating vs. steady-state).
- Simultaneous events require an explicit priority convention; departure-before-arrival is standard.
- Hand-tracing three events is enough to catch most logic errors before running code.

Next lecture: Manual tracing and performance measure computation (L07).